

Unidad 2: SendMessage

Se desea implementar una aplicación Android que permita al usuario enviar un mensaje desde una primera pantalla (*SendActivity*) a otra segunda pantalla (*ViewActivity*). Para ello se deberán aplicar los conocimientos de configuración del entorno, gestión de proyectos, depuración, control de versiones, y especialmente la organización de la interfaz con *Activities* y el diseño de interfaces XML con *LinearLayout*.

TAREA 1: Configuración y entorno

- **Ajuste de preferencias del IDE:** Para facilitar la programación, configura el autocompletado habilitando la ventana emergente automática de JavaDoc. Esto se realiza accediendo a *Settings > Editor > General > Code Completion*.
- **Integración de Plugins:** Revisa las notificaciones de actualización de plugins en el IDE para instalar o actualizar los complementos recomendados para el curso, tales como "**Atom Material Icons**", "**Rainbow Brackets**" y "**SonarQube for IDE**" (SonarLint).CD2.2 Gestiona emuladores y uso de ADB.
- Verifica comunicación con el emulador usando ADB.
 - Para consultar todos los emuladores o dispositivos conectados, ejecuta *adb devices -l*. Si la conexión es exitosa, su estado aparecerá como device.
 - Para acceder a la consola/shell del emulador y navegar por sus carpetas, utiliza el comando *adb shell*.

TAREA 2. Estructura y recursos del proyecto

Icono de la aplicación (Launcher):

- **Búsqueda del recurso:** Buscad un icono o imagen adecuada desde un repositorio gratuito recomendado, como Material Symbols de Google, Flaticon o Pixabay.
- **Genera el icono:**
 - Herramientas online: **Android Asset Studio** (permite generar iconos genéricos a partir de archivos PNG o SVG, descargando un .zip con todas las carpetas de densidad) o **NativeScript Image Builder**.
 - O bien en Android Studio: Haz clic derecho sobre la carpeta */res/mipmap/* (o directamente sobre la carpeta *app*) y selecciona **New > Image Asset**. Utiliza el asistente para cargar tu imagen fuente y generar automáticamente los iconos *ic_launcher* en todas las densidades necesarias.

Imagen vectorial (Vector Asset):

- La segunda actividad debe tener una imagen vectorial, haced clic derecho sobre la carpeta */res/drawable/* y seleccionad **New > Vector Asset**. Podéis elegir un icono de la galería integrada (por ejemplo, un icono de mensaje o confirmación). El entorno generará automáticamente el archivo de código XML vectorial, dejándolo listo para ser usado en vuestra interfaz gráfica.

TAREA 3: Diseño de la interfaz

El diseño de la aplicación seguirá el principio de separación entre la interfaz gráfica y el código lógico, utilizando archivos declarativos XML ubicados en el directorio `/res/layout/`.

- **Jerarquía de Vistas (View y ViewGroup):** La estructura base de las pantallas se definirá mediante contenedores como *LinearLayout*, al cual se le aplicarán parámetros de diseño (atributos) como *android:orientation="vertical"*.
- **Widgets e Identificadores:**
 - En la primera pantalla (*SendActivity*), se incorporarán elementos visuales (Views) como *TextView* para mostrar textos, un campo de entrada y un *Button* para capturar interacciones.
 - En la segunda pantalla (*ViewActivity*), además del *TextView* para mostrar el texto recibido, **deberéis incluir un componente *ImageView* que muestre el recurso vectorial creado en la Tarea 2.**
 - A cada widget interactivo se le asignará un atributo *android:id* (por ejemplo, *android:id="@+id/btnSend"*). Este ID generará automáticamente un número entero en el archivo local *R.java*, identificando el componente de forma única.
- **Recursos:** Las cadenas, los colores y tamaños deben estar en su correspondiente fichero (*strings.xml*, *colors.xml*, *dimens.xml*...).

TAREA 4. Lógica de la aplicación

- Asigna los elementos de la interfaz (*EditText*, *Button*) a variables con *findViewById*.

Flujo de la aplicación:

1. Recupera el texto introducido por el usuario en el campo de entrada.
2. Implementa un *OnClickListener* en el botón que cree un *Intent* y abra la segunda *Activity*, enviando el mensaje como extra.
3. En la segunda *Activity*, recibe el mensaje mediante el *Intent* y muéstralo en un *TextView*.
4. En la segunda *Activity*, se aplica color y se personaliza la fuente siguiendo el mismo estilo que la actividad de origen o *SendMessageActivity*,

Ciclo de Vida

- Agrupar los métodos que hacen referencia al ciclo de vida de una actividad en una región del IDE.

Depuración y ejecución: Desarrolla la lógica básica en Kotlin.

- Ejecuta la aplicación en el emulador comprobando que la transición entre *Activities* funciona correctamente.
- Usa la clase *android.util.Log* (por ejemplo, *Log.d()*) definiendo una constante TAG personalizada para registrar los estados del ciclo de vida.
- Usa puntos de interrupción o *breakpoints* para verificar el ciclo de vida de las *Activities* (*onCreate*, *onStart*, *onResume*, etc.).

TAREA 5: Comentarios en el código

Comenta el código que has modificado. Los comentarios en Kotlin deben contener etiquetas de estilo HTML como por ejemplo `<ul>`, `<b>`, enlaces externos y parámetros Javadoc o Kdoc como `@see`, `@autor...`

Se debe:

- Conocer la sintaxis de comentarios KDoc en Kotlin
- Usar etiquetas HTML para formatear documentación
- Uso de enlaces para comprender el uso de las clases utilizadas.
- Aplicar parámetros `@param`, `@see`, `@author`, `@version`
- Documentar ficheros XML con comentarios
- Mejorar la legibilidad y mantenibilidad del código

TAREA 6: Generar la documentación con Dokka

Se debe generar la documentación del proyecto en formato HTML en una carpeta llamada `/docs`

- Añadir plugin de Dokka dentro del catálogo de versiones oficial (`gradle/libs.versions.toml`)
- Conocer Dokka como generador de documentación
- Ejecutar tareas de Gradle desde terminal
- Interpretar documentación generada automáticamente
- Verificar que los comentarios **KDoc** se reflejan en la documentación

TAREA7: Documentación y comentarios

- Documenta el código con comentarios KDoc.
- **README.md** en GitHub con:
 - Capturas de pantalla de la aplicación en ejecución en el emulador. **OBLIGATORIO.**
 - Explicación de la estructura del proyecto y decisiones de diseño.
 - Descripción del proceso de depuración y evidencias de **Logcat** mediante una imagen. **OBLIGATORIO.**
 - Enlaces a la documentación oficial de Android Developer.
 - Imagen mostrando la conexión al director `/data/data/` de la aplicación.
- **CHANGELOG.md** documentando al menos dos versiones: v0.1 (configuración inicial y pantallas) y v1.0 (paso de datos completado y documentación generada).
- Archivo **MANUAL_USUARIO.md**, redactado en lenguaje sencillo (sin tecnicismos), explicando en 3 pasos cómo redactar y enviar un mensaje desde la app.

TAREA 8: Generación del APK firmado

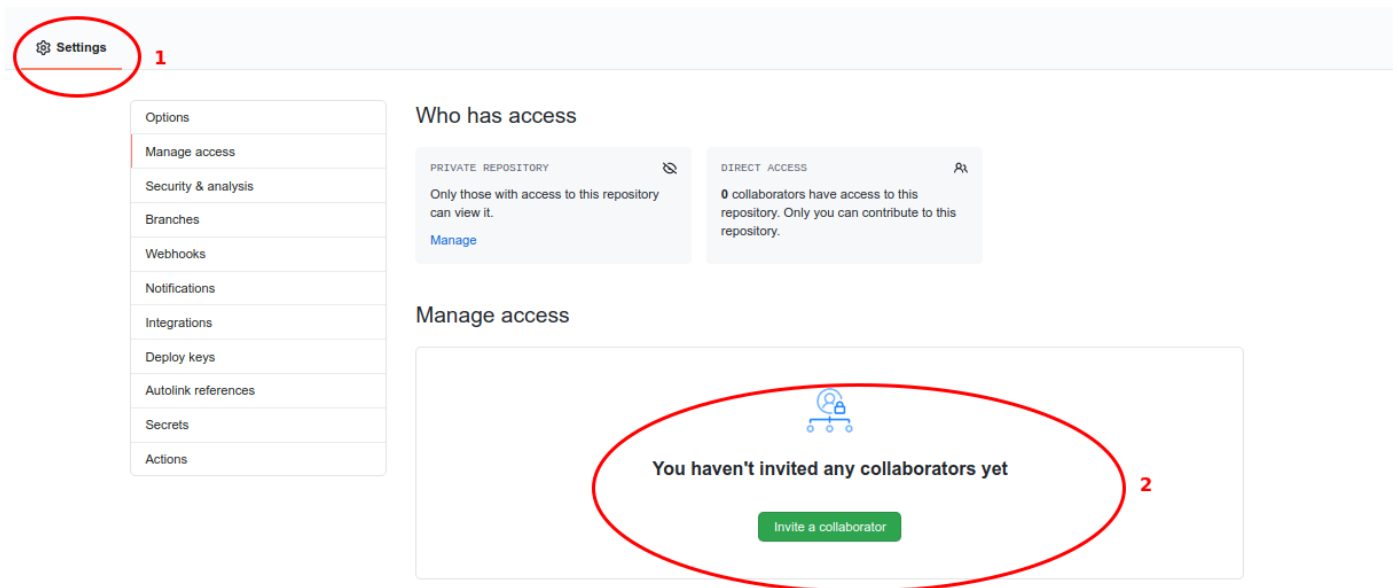
Para poder distribuir nuestra aplicación fuera del entorno de desarrollo y que pueda ser instalada en cualquier dispositivo real, necesitamos generar un archivo ejecutable (APK) firmado digitalmente en modo comercial o *reléase*.

Comprueba que el fichero generado lo subes a la raíz del proyecto de forma que el fichero llamado `app-release.apk` es visible.

TAREA 9: Publicación en GitHub

Debes crear una cuenta en la plataforma [GitHub](https://github.com). A continuación, crea un repositorio **privado** con el nombre **"HelloWorld"** que contenga el proyecto que se ha realizado en clase. Tenéis que añadir mi usuario **@moronlu18** como colaborador, para ello tenéis que realizar las siguientes operaciones:

1. Selecciona el repositorio con nombre **HelloWorld**
2. Selecciona la opción **Settings**
3. Selecciona la opción **Manage access**.
4. Si el repositorio es privado, en la opción "**invite a Collaborator**" busca el usuario para comprobar que existe en GitHub (@moronlu18) o inserta mi correo (lourdes@iesportada.org) para enviar un correo.



Cuando tengas el repositorio creado en GitHub:

- Sincroniza repositorio local y remoto con operaciones básicas (*pull, fetch, merge*).
- Aplica correctamente un archivo *gitignore*

TAREA 9: Generación de la documentación GitHub Page

Utiliza Jekyll para generar la documentación estática del proyecto en GitHub. Comprueba que se ha generado en. `github/workflows` un fichero llamado **jekyll-gh-pages.yml** que contiene la configuración de la herramienta en el repositorio.

REQUISITOS DE ENTREGA MOODLE

DOCUMENTACIÓN PROYECTO GITHUB

Entrega un fichero PDF con el nombre: **AEA01_Nombre_Apellidos.pdf**. Este documento debe contener:

1. Tu nombre de usuario de GitHub.
2. El enlace al repositorio creado.

PROYECTO REALIZADO

Se debe entregar el proyecto en un fichero comprimido cuyo nombre debe tener el siguiente formato: **AEA01_Nombre_Apellidos.zip**.

Antes de entregar el proyecto, debes realizar los siguientes pasos: *Menú: Build → Clean Project*. Esta opción elimina los archivos compilados temporalmente

Nota: Si no se cumplen estos requisitos, el ejercicio se evaluará como NO EVALUADO (NE).